ASSESSING THE SECURITY OF CERTIFICATES AT SCALE

David McGrew, Brandon Enright, Andrew Chi

August 12, 2023

DEFCON 31

THE WEAK ENTROPY/KEY PROBLEM

2011 Bernstein. Fast multiplication and its applications, *Algorithmic Number Theory*, Volume 44.

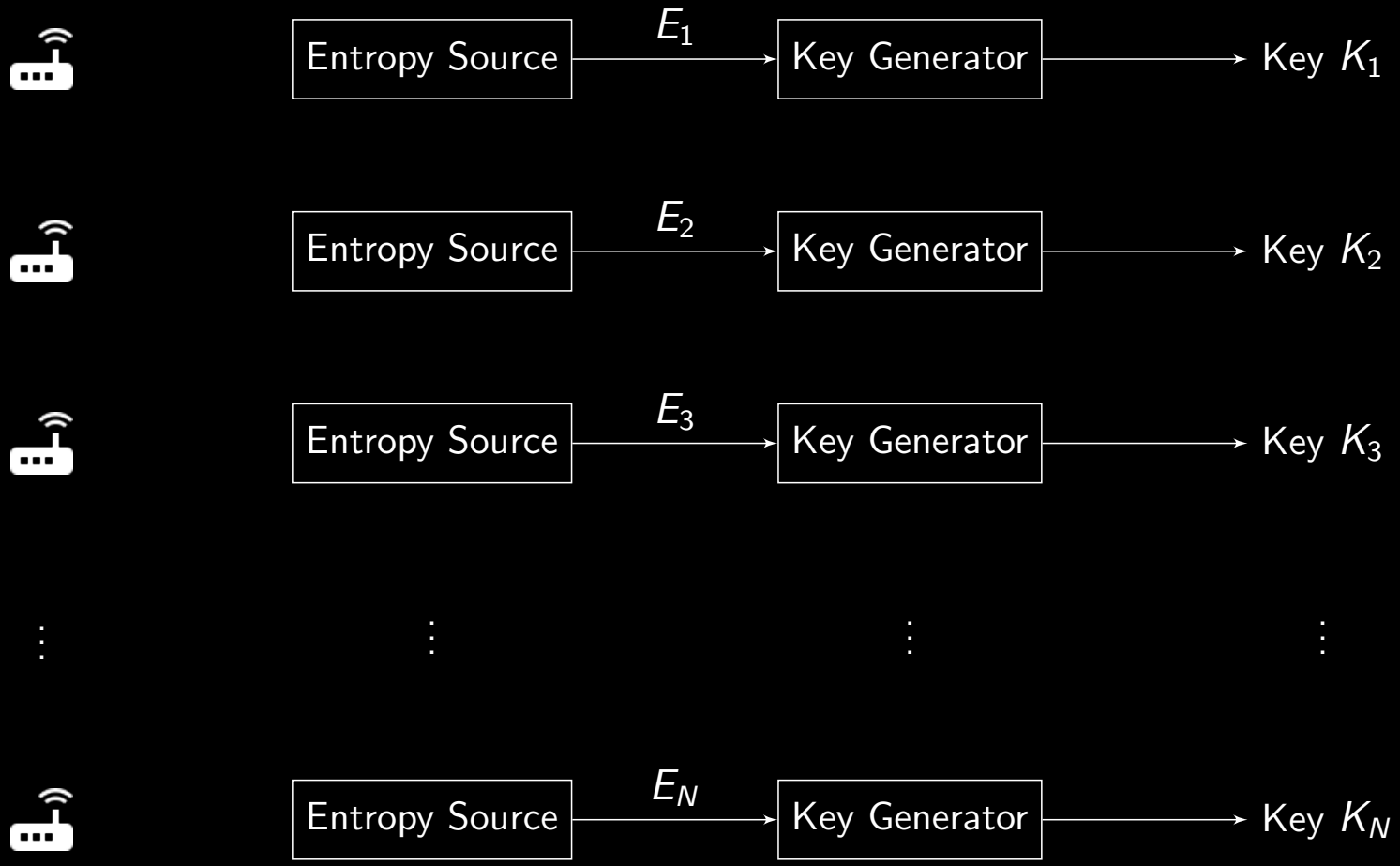
2012 Heninger, Durumeric, Wustrow, and Halderman. Mining your ps and qs: Detection of widespread weak keys in network devices. In *USENIX Security 2011*.

2016 Hastings, Fried, and Heninger. Weak keys remain widespread in network devices. In *ACM Internet Measurement Conference 2016*.

2019 Kilgallin and Vasko. Factoring RSA Keys in the IoT Era. In *IEEE International Conference on Trust, Privacy and Security in Intelligent Systems and Applications*.

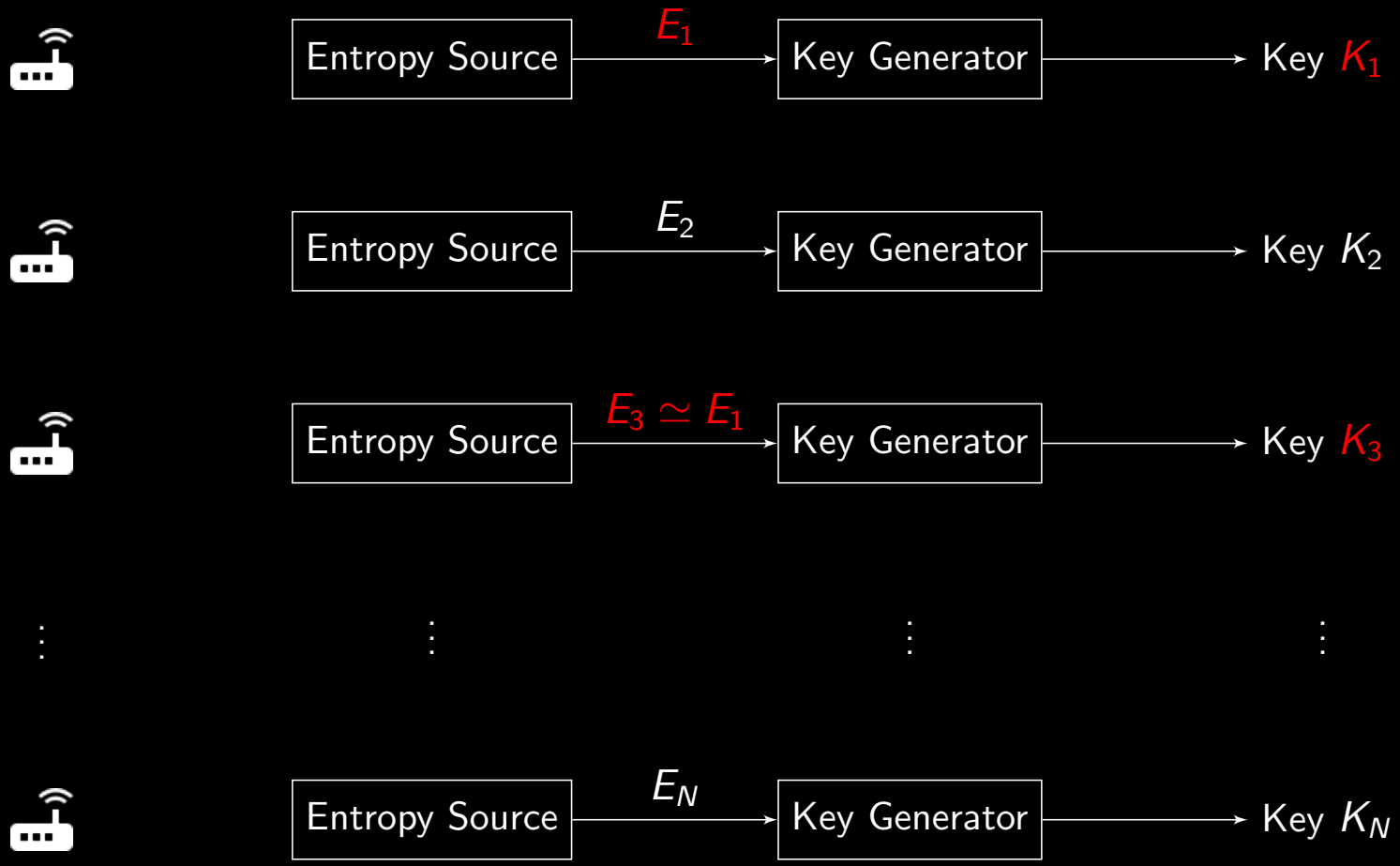
2023 Migration to Quantum-Safe Cryptography

KEY GENERATION



WEAK KEY GENERATION

WEAK KEY GENERATION



FAILURE MODES OF PUBLIC KEY ALGORITHMS

Common Keys

- ECC, RSA, ...
- Detection: identical keys generated by distinct devices
- Looks like key reenrollment

Common Factors

- RSA
- Detection: Batch GCD
- Strong evidence of failure

RSA

$$M \quad = \quad P \cdot Q$$

Public Key — Private Key

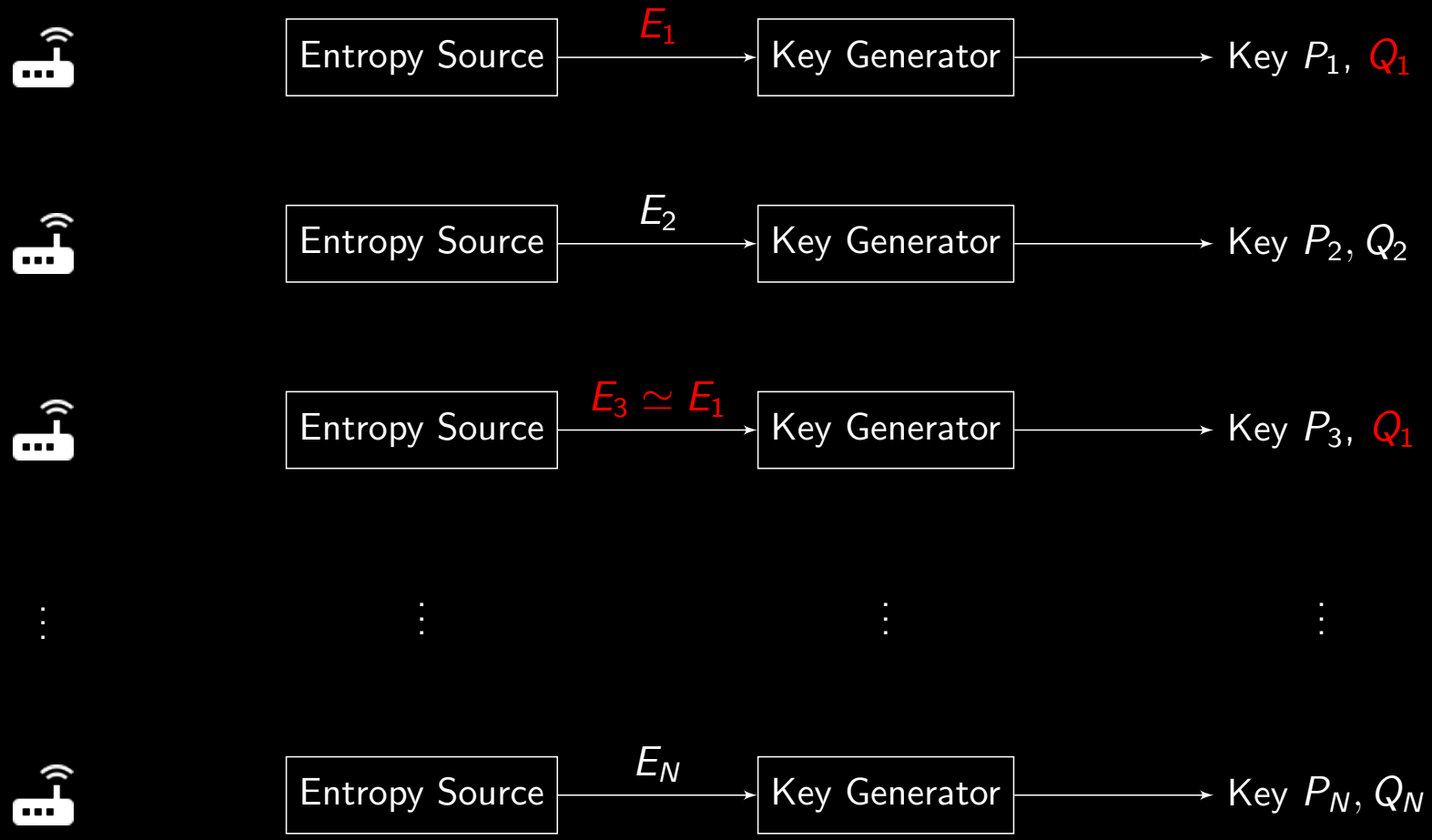
$\leftarrow$ Multiplication is **easy**

Factoring is **hard** $\rightarrow$

M: n-bit integer

P, Q: random $n/2$-bit primes

RSA COMMON FACTORS



FINDING COMMON FACTORS WITH BATCH GCD

$$\begin{aligned}\mathrm{GCD}(M_1, M_3) &= \mathrm{GCD}(P_1 \cdot Q_1, P_3 \cdot Q_1) \\ &= Q_1\end{aligned}$$

GCD is **easy**

Batch GCD computes GCD of each pair (M_i, M_j)

OBTAINING CERTIFICATES

Scans	Monitoring	CA Logs
Active	Passive	Passive
`tls_scanner`	`mercury`	`cert_analyze`

Tools are BSD-licensed Linux/POSIX CLI
BGCD and ASN.1 parsers implemented in C++17

TLS SCAN TOOL

`tls_scanner`

`--host-file` *`hosts`* scans all hosts in the file *`hosts`*.

`--write-certs` writes out certificates as PEM `CERTIFICATE` files

Obtains certs from a set of hosts to be audited

CERTIFICATE TOOL

`cert_analyze`

`--filter regex=`*rgx* filters certs against *rgx*.

`--pem-output` writes out (filtered) certificates in PEM format

`--sha1-output` writes out SHA1 fingerprint of certificate

Selects a (manageably small) subset of certs from similar devices

FREQUENT STRINGS IN FACTORABLE CERTIFICATES

Issuer: Organization

```
TPLINK
DrayTek Corp.
Cisco-Linksys, LLC
HTTPS Mgmt Cert for
SonicWALL
Tridium
Netgear Inc.
Cisco Small Business
D-Link|D-LINK
SAMSUNG
Technicolor
Honeywell
Linksys International Inc.
Fortinet Ltd.
```

See references for other common strings

Some other certificate fields are useful indicators

BATCH GCD TOOL

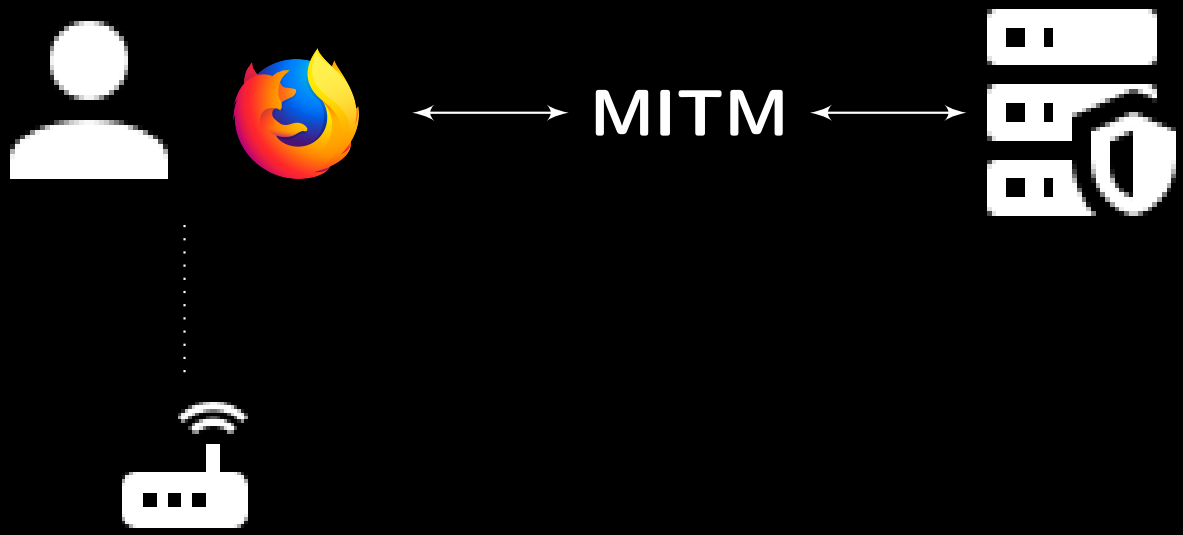
`batch_gcd`

`--cert-file` *`filename`* Reads certificates from *`filename`* in `CERTIFICATE` format

`--write-keys` For each factored RSA key, writes the private key in `RSA PRIVATE KEY` format as *`filename-number`*`.rsapriv.pem` and the corresponding cert as *`filename-number`*`.cert.pem`

Two passes over input, to minimize RAM usage

DEMONSTRATION: MACHINE IN THE MIDDLE ATTACK



Acknowledgment: https://mitmproxy.org

CONCLUSIONS

- Batch testing is practical and effective
- Future work:
  - Detect more failure modes
  - Analyze manufacturing certificates

Download tools and documentation:

https://github.com/cisco/mercury

THANK YOU

ABSTRACT

The security of digital certificates is too often undermined by the use of poor entropy sources in key generation. Flawed entropy can be hard to discover, especially when analyzing individual devices. However, some flaws can be detected when a large set of keys from the same entropy source are analyzed, as was dramatically demonstrated in 2012 and 2016 by the detection of weak HTTPS keys on the Internet.

In this talk, we present tools and techniques to identify weak keys at scale, by checking issued certificates obtained from passive monitoring, active network scans, or certificate authority logs. Our tools use efficient multithreaded implementations of network monitors, scanners, certificate parsers, and mathematical tests. The batch greatest common divisor test (BGCD) identifies RSA public keys with common factors, and outputs the corresponding private keys. The common key test identifies distinct devices that share identical keys. We report on findings from both tests and demonstrate how to audit HTTPS servers, run BGCD on 100M+ keys, identify RSA keys with common factors, and generate the corresponding private keys. Because nothing convinces like an attack, we show how to produce and use PEM files for factored keys.

`https://forum.defcon.org/node/245774`